
Sparkle: LLM-Driven Formal Hardware Synthesis from Natural Language to Silicon

Anonymous Author(s)

Affiliation

Address

email

Abstract

1

2 1 Introduction

3 Hardware design remains one of the most error-prone and labor-intensive disciplines in comput-
4 ing. A single bit-width mismatch or missing case in a finite state machine can propagate silently
5 through synthesis, only to surface weeks later as a silicon bug costing millions of dollars to fix.
6 Despite decades of progress in electronic design automation (EDA), the dominant hardware descrip-
7 tion languages—Verilog and VHDL—still lack the static guarantees that modern software engineers
8 take for granted: dependent types, exhaustive pattern matching, and machine-checked proofs of
9 correctness.

10 Two largely independent research threads have sought to address this gap. On one hand, *type-safe*
11 *HDLs* such as Chisel [Bachrach et al., 2012], Bluespec [Nikhil, 2004], and Clash [Baaib et al., 2010]
12 embed hardware descriptions in host languages with richer type systems, eliminating entire classes
13 of bugs at compile time. On the other hand, *large language models* (LLMs) have demonstrated
14 remarkable ability to generate Verilog from natural language specifications [Thakur et al., 2023,
15 Liu et al., 2023], promising to democratize hardware design by lowering the barrier to entry. Yet
16 these two threads remain disconnected: LLM-generated Verilog inherits all the pitfalls of the target
17 language—unintended latches, combinational loops, and width mismatches—while type-safe HDLs
18 still require expert knowledge to use effectively.

19 We argue that these threads are not merely complementary but *synergistic*: a dependently-typed
20 HDL provides exactly the kind of rich, structured feedback that an LLM agent needs to iteratively
21 converge on correct hardware. In this paper, we present SPARKLE, a framework that unifies LLM-
22 driven code generation with formal hardware verification by targeting a hardware DSL embedded
23 in Lean 4 [Moura and Ullrich, 2021]. Given a natural language specification, an LLM coding agent
24 generates hardware in Sparkle’s Signal DSL, where Lean’s type checker immediately catches width
25 mismatches, the monadic structure prevents combinational loops by construction, and exhaustive pat-
26 tern matching eliminates unintended latches. When compilation succeeds, the framework automati-
27 cally extracts synthesizable SystemVerilog, runs functional simulation against reference testbenches,
28 and performs physical synthesis through OpenROAD [Ajayi et al., 2019] with the SkyWater 130nm
29 PDK [Google and SkyWater Technology, 2020]. A closed-loop PPA optimization stage then feeds
30 power, performance, and area metrics back to the agent, enabling iterative architectural refinement
31 with formal equivalence guarantees.

32 The key insight behind SPARKLE is that *the compiler is the verifier*. Rather than generating Verilog
33 and hoping it is correct, the agent receives immediate, actionable feedback from Lean’s type system
34 at every iteration—feedback that is far more informative than the cryptic warnings of a Verilog linter.

35 This transforms the LLM’s generate-and-test loop from a stochastic search over syntactically valid
36 programs into a *type-guided refinement* process that systematically narrows the space of candidate
37 designs.

38 Our contributions are as follows:

- 39 • We introduce SPARKLE, the first framework that combines LLM-driven hardware generation with
40 a dependently-typed HDL, enabling type-guided iterative refinement from natural language to
41 synthesizable silicon.
- 42 • We design an end-to-end automated pipeline spanning code generation, formal type checking,
43 functional simulation, logic synthesis, and place-and-route, with a closed-loop PPA optimization
44 stage that preserves functional correctness.
- 45 • We demonstrate that targeting a type-safe HDL substantially improves LLM success rates com-
46 pared to direct Verilog generation, as compile-time type errors provide structured feedback that
47 guides the agent toward correct designs.
- 48 • We show that the framework can produce formally verified hardware—including machine-checked
49 equivalence proofs between specifications and optimized implementations—bridging the gap be-
50 tween LLM-generated code and provably correct hardware.

51 2 Related Work

52 **Hardware Description Languages.** The limitations of Verilog and VHDL have motivated a rich
53 line of work on higher-level HDLs. Chisel [Bachrach et al., 2012] embeds hardware construction
54 in Scala, providing parameterized generators and the FIRRTL intermediate representation that elim-
55 inates unintended latches through its lowering passes. Bluespec SystemVerilog [Nikhil, 2004] uses
56 guarded atomic actions to prevent race conditions by construction. Clash [Baaij et al., 2010] lever-
57 ages Haskell’s type system to encode hardware semantics, enabling equational reasoning about cir-
58 cuits. PyMTL [Lockhart et al., 2014] and MyHDL [Decaluwe, 2004] prioritize accessibility by
59 embedding hardware in Python. High-level synthesis (HLS) tools [Cong et al., 2011, Canis et al.,
60 2011] raise the abstraction further by compiling C/C++ to RTL, but sacrifice fine-grained control
61 over micro-architecture. While these approaches improve productivity, none provides the combi-
62 nation of dependent types, formal verification, and synthesizable extraction that Sparkle achieves
63 through its embedding in Lean 4.

64 **Formal Verification of Hardware.** Formal methods have a long history in hardware verification,
65 from model checking [Clarke et al., 2018] and bounded model checking [Biere, 2009] to equivalence
66 checking via tools like ABC [Brayton and Mishchenko, 2010]. These approaches typically operate
67 on the RTL or gate-level netlist *after* the design is written, treating verification as a separate post-
68 hoc phase. An alternative line of work embeds hardware in proof assistants to enable *verification by*
69 *construction*. Kami [Choi et al., 2017] provides a Bluespec-like language in Coq with modular re-
70 finement proofs. Silver Oak [Choi et al., 2021] generates verified hardware from Coq specifications.
71 ACL2 has been used extensively in industrial processor verification [Hunt et al., 2017]. Sparkle
72 follows this tradition but targets Lean 4 [Moura and Ullrich, 2021], whose metaprogramming ca-
73 pabilities enable seamless SystemVerilog extraction alongside formal proofs, and whose dependent
74 type system provides richer compile-time feedback than prior proof-assistant HDLs.

75 **LLMs for Hardware Design.** The application of LLMs to hardware design has gained signif-
76 icant momentum. Early work by Pearce et al. [2020] explored generating Verilog from English
77 descriptions. More recently, general-purpose code models such as Codex [Chen et al., 2021], Code
78 Llama [Rozière et al., 2023], and StarCoder 2 [Li et al., 2024] have been evaluated on Verilog
79 generation tasks. VeriGen [Thakur et al., 2024] fine-tunes LLMs specifically on Verilog corpora,
80 while Chip-Chat [Blocklove et al., 2023] and ChipGPT [Chang et al., 2024] explore conversational
81 and multi-step hardware design flows. Benchmarks such as VerilogEval [Liu et al., 2023] and
82 RTLLM [Lu et al., 2024] provide standardized evaluation suites. A common limitation across this
83 body of work is that LLMs generate Verilog directly, inheriting all of its pitfalls: the generated code
84 may contain width mismatches, unintended latches, or combinational loops that pass syntax checks
85 but fail in synthesis or silicon. Our work addresses this by redirecting the LLM to target a type-safe

86 HDL, where the compiler catches these errors immediately and provides structured feedback for
 87 iterative correction.

88 **LLM Agents for Code Generation.** Tool-augmented LLM agents have shown strong results in
 89 software engineering. ReAct [Yao et al., 2023] interleaves reasoning and action in a general agent
 90 framework. SWE-agent [Yang et al., 2024a] and SWE-bench [Jimenez et al., 2024] demonstrate that
 91 LLM agents can resolve real-world GitHub issues by reading code, editing files, and running tests
 92 in a feedback loop. Our coding agent follows a similar paradigm—iterative generation, compilation,
 93 and error-driven repair—but operates in the hardware domain, where the feedback signal from a
 94 dependently-typed compiler is substantially richer than typical software build errors.

95 **LLMs for Theorem Proving.** LLMs have also been applied to formal theorem proving, with sys-
 96 tems like GPT-f [Polu and Sutskever, 2020] and LeanDojo [Yang et al., 2024b] demonstrating the
 97 ability to generate proof steps in Lean. Our work is complementary: while these systems focus
 98 on proving mathematical theorems, Sparkle leverages Lean’s proof capabilities to verify hardware
 99 properties. The LLM agent in our framework can potentially generate not only hardware implemen-
 100 tations but also formal correctness proofs, a direction we explore through architecture equivalence
 101 verification.

102 3 Method

103 We present SPARKLE, an end-to-end framework that translates natural language hardware specifica-
 104 tions into formally verifiable, synthesizable hardware designs. The key insight is that by targeting
 105 a dependently-typed hardware description language embedded in Lean 4 rather than generating Ver-
 106 ilog directly, the LLM agent receives rich compile-time feedback—type errors, width mismatches,
 107 and formal verification failures—that guides iterative refinement toward correct-by-construction
 108 hardware.

109 3.1 Overview

110 Figure ?? illustrates the complete pipeline. Given a natural language specification $\mathcal{S}$ and an optional
 111 reference Verilog implementation $\mathcal{V}_{\text{ref}}$, our system produces a formally described hardware design
 112 through the following stages:

- 113 1. **LLM Code Generation.** A coding agent $\mathcal{A}$ translates $\mathcal{S}$ into Sparkle HDL (Lean 4) code $\mathcal{L}$,
 114 using few-shot examples from a verified benchmark library and iterative error correction.
- 115 2. **Compilation & Extraction.** The Lean 4 compiler type-checks $\mathcal{L}$ and extracts synthesizable
 116 SystemVerilog $\mathcal{V}$ via the `#synthesizeVerilog` metaprogram.
- 117 3. **Functional Verification.** $\mathcal{V}$ is simulated against reference testbenches to verify functional cor-
 118 rectness.
- 119 4. **Physical Synthesis.** $\mathcal{V}$ is synthesized and placed-and-routed using OpenROAD with the SkyWa-
 120 ter 130nm PDK, yielding PPA (Power, Performance, Area) metrics.
- 121 5. **PPA Optimization.** The agent receives PPA feedback and iteratively optimizes the architecture
 122 while preserving functional correctness.

123 3.2 Sparkle HDL: Hardware in Lean 4

124 Sparkle HDL is a domain-specific language embedded in Lean 4 that describes hardware using
 125 Signal combinators. A signal `Signal dom  $\alpha$` represents a synchronous stream of values of type
 126 α under clock domain `dom`. Semantically, a signal is a function from discrete time steps to values:
 127 `Signal dom  $\alpha \triangleq \mathbb{N} \rightarrow \alpha$` . This denotational semantics enables both cycle-accurate simulation
 128 (by evaluating the function) and formal reasoning (by proving properties over the function). The
 129 embedding in Lean 4 provides four properties that make it uniquely suited as an LLM compilation
 130 target for chip architecture.

131 **Type-Safe Bit Widths.** Lean 4’s dependent type system encodes bit widths at the type level via
 132 `BitVec  $N$` . Width mismatches—a pervasive source of silent bugs in Verilog—become compile-
 133 time errors. For example, adding a 4-bit and an 8-bit signal without explicit extension is rejected

by the type checker, providing immediate feedback to the LLM agent. This is critical for chip architecture: a datapath width error in a pipelined multiplier or an address bus mismatch in a memory controller would silently propagate through Verilog synthesis but is caught *before compilation completes* in Sparkle.

Structural Safety via the Signal Monad. Combinational logic forms a directed acyclic graph enforced by the Signal monad; state feedback is only possible through explicit register primitives (`Signal.register`, `Signal.loop`), making combinational loops impossible by construction. Lean’s exhaustive pattern matching prevents unintended latches—a common synthesis hazard in Verilog’s `always` blocks. For sequential circuits, Sparkle provides an imperative `Signal.circuit` macro that desugars into `Signal.loop` and `Signal.register` calls, enabling a natural description of pipelines and state machines:

```
def counter {dom : DomainConfig} : Signal dom (BitVec 8) :=
  Signal.circuit do
    let count <- Signal.reg 0#8
    count <- count + 1#8
    return count
```

This compiles to a single `always_ff` block with proper clock and reset handling. More complex architectures—finite state machines, multi-stage pipelines, memory-mapped controllers—compose from the same primitives. For instance, an FSM is expressed as a `Signal.loop` over state with `Signal.mux` for transition logic, where the type system ensures all state encodings have consistent bit widths.

Memory and Datapath Primitives. Sparkle provides synthesizable memory primitives that map directly to SRAM/BRAM in physical synthesis: `Signal.memory` (synchronous read with 1-cycle latency), `Signal.memoryComboRead` (combinational read for register files), and `Signal.memoryWithInit` (pre-loaded ROM/RAM). Address and data widths are encoded in the type signature, so a memory access with a mismatched address width is a compile-time error. Combined with the standard arithmetic operators overloaded on `Signal dom (BitVec N)`, these primitives allow the LLM agent to describe complete chip architectures—ALUs, register files, caches, and interconnects—within a single type-checked Lean 4 file.

Formal Verification. Designers (or the LLM agent) can state and prove theorems about the hardware directly in Lean 4. Because signals have a denotational semantics as functions over time, properties can be stated as standard Lean theorems. For instance, a counter’s initial value and increment behavior:

```
theorem counter_starts_at_zero :
  (counter.atTime 0) = 0#8 := by rfl

theorem counter_increments (n : Nat) :
  (counter.atTime (n+1)) = (counter.atTime n) + 1#8 := by
  simp [counter, Signal.register]; rfl
```

For architecture optimization, equivalence proofs between a specification and an optimized implementation provide machine-checked correctness certificates:

```
theorem equiv (input : Signal dom (BitVec 3)) :
  optimized input = spec input := by
  unfold optimized spec; rfl
```

This capability is essential for the PPA optimization loop (§3.6), where the agent must ensure that architectural transformations preserve functional correctness across all possible inputs—a guarantee that simulation-based testing alone cannot provide.

3.3 Compilation and Verilog Extraction

A central design goal of Sparkle is that the *compiler is the verifier*: every Lean 4 build invocation simultaneously type-checks the hardware description, enforces structural invariants, and extracts synthesizable RTL. This section details the compilation pipeline that transforms a Lean 4 source file into backend-ready SystemVerilog.

186 **Lean 4 Type Checking.** When the LLM agent writes a Sparkle design to
187 `Generated/{prob_id}.lean` and invokes `lake build`, Lean’s elaborator resolves all de-
188 pendent types, verifies bit-width consistency across the entire design, and checks that all pattern
189 matches are exhaustive. A single `lake build` invocation thus catches three classes of bugs
190 that would silently pass Verilog synthesis: (1) width mismatches between connected signals,
191 (2) incomplete case analysis in multiplexer or FSM logic, and (3) type-level violations such as using
192 a boolean where a bit vector is expected. The compiler error messages are structured and localized,
193 enabling the LLM agent to apply targeted fixes rather than guessing.

194 **Intermediate Representation.** Upon successful type checking, the `#synthesizeVerilog`
195 metaprogram lowers the `Signal` combinator graph into Sparkle’s intermediate representation (IR).
196 The IR is a netlist-like structure that preserves the hierarchical module boundaries and signal names
197 from the Lean source, maintaining a 1:1 structural correspondence between the Lean code and the
198 generated Verilog. Unlike Chisel’s FIRRTL, which aggressively transforms and renames signals
199 during lowering, Sparkle’s IR retains human-readable names, making the generated SystemVerilog
200 inspectable by both the LLM agent and human designers.

201 **Design Rule Check (DRC).** Before Verilog emission, the IR passes through a built-in Design
202 Rule Check that enforces backend-friendly RTL conventions inspired by industry standards (e.g.,
203 STARC guidelines). The primary check is *registered output enforcement*: the DRC verifies that
204 all module outputs are driven directly by flip-flops rather than combinational logic. Unregistered
205 outputs cause critical path explosion across module boundaries and make static timing analysis
206 (STA) unpredictable. When the DRC detects a violation, it emits a structured warning:

207 [DRC] Module 'alu': output 'out' is not driven by a register

208 This feedback is actionable for the LLM agent: it can insert a `Signal.register` at the output
209 to resolve the violation. The DRC thus serves as an early physical-awareness check, reducing the
210 number of synthesis-P&R iterations needed to achieve timing closure.

211 **SystemVerilog Emission.** The final stage emits clean, synthesizable SystemVerilog from the
212 DRC-validated IR. Clock and reset signals are automatically inserted into the module port list. Reg-
213 isters are emitted as `always_ff` blocks with synchronous reset. Combinational logic is emitted as
214 `assign` statements or `always_comb` blocks. The generated code is compatible with all major EDA
215 tools (Yosys, Synopsys Design Compiler, Cadence Genus) and simulation tools (Verilator, Icarus
216 Verilog). A `TopModule` wrapper is automatically generated to map Sparkle’s port naming conven-
217 tions to standard testbench interfaces, enabling seamless integration with the VerilogEval reference
218 testbenches used in our evaluation pipeline.

219 3.4 LLM Coding Agent

220 The coding agent $\mathcal{A}$ is an LLM equipped with tool-use capabilities that iteratively writes, compiles,
221 and debugs Sparkle HDL code. We instantiate $\mathcal{A}$ using Claude with the following components:

222 **System Prompt.** The agent receives a structured system prompt containing: (1) the Sparkle HDL
223 syntax reference, including the `Signal` API, type system, and core operators; (2) design patterns
224 for combinational logic, sequential circuits, and finite state machines; (3) critical rules encoding
225 common pitfalls (e.g., “use `Signal.mux` instead of `if-then-else`”); and (4) a prescribed workflow
226 for iterative development.

227 **Tool Suite.** The agent has access to seven tools: `bash` (execute shell commands with 120s
228 timeout), `read_file`, `write_file`, `edit_file` (surgical string replacement), `grep`, `glob`, and
229 `list_directory`. These tools enable the agent to explore the codebase, read benchmark examples,
230 write Lean code, invoke the compiler, and analyze error messages.

231 **Few-Shot Retrieval.** Before generating code, the agent reads from a library of 10 verified bench-
232 mark implementations spanning combinational circuits, sequential logic, and FSMs. These serve as
233 in-context examples that demonstrate correct Sparkle HDL patterns and naming conventions.

234 **Iterative Compile-Fix Loop.** The agent follows a multi-turn conversation protocol:

- 235 1. Read the natural language specification and reference Verilog.
- 236 2. Identify similar benchmark examples via `grep/glob`.
- 237 3. Write an initial Lean 4 implementation to `Generated/{prob_id}.lean`.
- 238 4. Invoke `lake build` and parse compiler diagnostics.
- 239 5. If compilation fails, analyze the error (type mismatch, unknown identifier, syntax error), apply a
- 240 targeted fix via `edit_file`, and retry.
- 241 6. Once compilation succeeds, verify the extracted SystemVerilog.

242 The agent is allowed up to T turns (default $T=40$) per problem. Each turn consists of one LLM
243 inference followed by zero or more tool invocations.

244 3.5 Automated Evaluation Pipeline

245 Once the agent produces a candidate Lean 4 file, the evaluation pipeline assesses it through a se-
246 quence of increasingly stringent checks:

- 247 1. **Lean Compilation.** `lake build Generated.{prob_id}` verifies type correctness and trig-
248 gers Verilog extraction.
- 249 2. **SystemVerilog Extraction.** The build output is parsed to extract the generated `.sv` file. A
250 `TopModule` wrapper is automatically generated to map Sparkle’s port naming conventions to the
251 reference testbench interface.
- 252 3. **Lint Check.** Verilator or Icarus Verilog performs static analysis to catch synthesis-incompatible
253 constructs.
- 254 4. **Functional Simulation.** The design is simulated against the VerilogEval reference testbench
255 using Icarus Verilog. The testbench applies input vectors and compares outputs cycle-by-cycle,
256 reporting the number of mismatches out of total samples.
- 257 5. **Logic Synthesis.** Designs passing simulation are synthesized using Yosys targeting the SkyWater
258 130nm HD standard cell library (`sky130_fd_sc_hd`) via the OpenROAD Flow Scripts (ORFS)
259 Docker environment. This yields gate-level netlists and initial PPA estimates.
- 260 6. **Place & Route.** The synthesized netlist undergoes floorplanning, placement, clock tree synthesis,
261 routing, and finishing in OpenROAD, producing a GDSII layout. Static timing analysis (STA)
262 reports worst negative slack (WNS) across multiple PVT corners (SS/TT/FF).

263 The pipeline is fully automated and containerized: synthesis and P&R run inside a Docker container
264 with pre-installed ORFS and the sky130 PDK, ensuring reproducibility.

265 3.6 PPA Optimization Loop

266 For designs that pass functional simulation and synthesis, we employ a closed-loop PPA optimiza-
267 tion strategy that is one of the central contributions of this work. The key idea is that by coupling the
268 LLM agent with quantitative physical design feedback, we transform hardware optimization from
269 a manual, expert-driven process into an automated, iterative refinement loop. The agent receives
270 concrete metrics about the physical characteristics of its design and uses this information to make
271 informed architectural decisions.

272 **PPA Metrics.** After each synthesis and place-and-route iteration, the pipeline extracts a compre-
273 hensive PPA vector $\mathbf{p}_i = (\text{area}_i, \text{power}_i, \text{WNS}_i, \text{cells}_i)$ from the OpenROAD reports. Area is mea-
274 sured in μm^2 from the Yosys synthesis statistics (e.g., “Chip area for module: 45.04 μm^2 ”), decom-
275 posed into sequential and combinational contributions. Power is reported in μW from OpenROAD’s
276 power analysis across multiple PVT corners. Worst Negative Slack (WNS) is extracted from static
277 timing analysis across slow-slow (SS), typical-typical (TT), and fast-fast (FF) corners, indicating
278 how far the design is from meeting timing at the target clock frequency. Cell count provides a proxy
279 for design complexity and routing congestion.

280 **Feedback Construction.** At each optimization iteration i , a structured feedback message is con-
281 structed for the agent containing: (1) the current PPA metrics $\mathbf{p}_i$ with human-readable formatting;

(2) the complete optimization history $\{\mathbf{p}_0, \dots, \mathbf{p}_i\}$ so the agent can identify trends; (3) a delta analysis highlighting which metrics improved or degraded relative to the previous iteration; and (4) targeted optimization guidance based on the dominant bottleneck. For area-dominated designs, the guidance suggests reducing intermediate signals, merging operations, or using more compact encodings. For timing-critical designs, it suggests pipelining long combinational paths or restructuring mux chains. This structured feedback transforms raw EDA tool output into actionable intelligence that the LLM can reason about.

Optimization Procedure. Let $\mathbf{p}_i$ denote the PPA vector after iteration i . The optimization proceeds for K iterations (default $K=3$):

1. **Agent Resume.** The agent resumes from its prior conversation state with the PPA feedback appended, preserving full context of the design decisions made so far. This is critical: the agent retains knowledge of the Sparkle HDL patterns it used, the type errors it resolved, and the architectural choices it made, enabling informed optimization rather than blind search.
2. **Architecture Modification.** The agent modifies the Lean 4 source to produce an optimized variant. Common transformations include: replacing parallel datapaths with serialized versions to reduce area, merging cascaded multiplexers into lookup tables, eliminating redundant registers, and restructuring arithmetic expressions to reduce critical path depth.
3. **Re-compilation and Type Checking.** The modified design is re-compiled through Lean 4, ensuring that the optimization did not introduce type errors, width mismatches, or structural violations. This is a key advantage over optimizing Verilog directly: the type system catches optimization-induced bugs immediately.
4. **Re-evaluation.** The design passes through the full pipeline (compilation $\rightarrow$ simulation $\rightarrow$ synthesis $\rightarrow$ P&R), producing updated PPA metrics $\mathbf{p}_{i+1}$.
5. **Improvement Assessment.** An iteration is considered an improvement if any metric improves by $>5\%$ without any metric degrading by $>20\%$:

$$\text{improved}(\mathbf{p}^*, \mathbf{p}_{i+1}) = \exists m : \frac{p_m^* - p_{i+1,m}}{|p_m^*|} > 0.05 \wedge \forall m : \frac{p_{i+1,m} - p_m^*}{|p_m^*|} < 0.20 \quad (1)$$

where $\mathbf{p}^*$ is the global best PPA vector across all iterations. This asymmetric threshold allows the agent to explore trade-offs (e.g., slightly increasing area to significantly improve timing) while preventing catastrophic regressions.

6. **Rollback on Failure.** If simulation fails after an optimization attempt—indicating the transformation broke functional correctness—the Lean source is rolled back to the pre-iteration version and the loop continues with the next iteration, rather than terminating early. This exhaustive strategy ensures the agent explores the full optimization budget.

After all K iterations complete, the globally best design (by PPA) is restored as the final output. The entire loop is fully automated: no human intervention is required between iterations.

Formal Equivalence in the Optimization Loop. A unique capability enabled by Sparkle’s embedding in Lean 4 is that the agent can produce formal equivalence proofs between the original and optimized architectures as part of the optimization loop. After each architectural transformation, the agent can attempt to prove:

```
theorem opt_equiv (input : Signal dom (BitVec N)) :
  optimized_v2 input = original input := by
  unfold optimized_v2 original; simp [Signal.mux, ...]; rfl
```

When such a proof succeeds, it provides a machine-checked guarantee that the optimization preserves functional correctness for *all* possible inputs—not just the test vectors in the simulation test-bench. This is particularly valuable for optimizations that restructure control logic or change state encodings, where simulation coverage may be insufficient to catch corner-case divergences.

3.7 Architecture Exploration

For designs where the optimization space is large, we support an architecture exploration mode. The agent generates C distinct architectural candidates (default $C=3$), each implementing the same

specification with different micro-architectural choices (e.g., parallel vs. serial datapaths, different encoding schemes, tree vs. chain reduction). Each candidate is independently evaluated through the full pipeline, and the best is selected by PPA metrics.

When formal verification is enabled, the agent additionally produces equivalence proofs between the original specification and each optimized architecture:

```
theorem arch_equiv (input : Signal dom (BitVec N)) :
  arch_optimized input = spec input := by
  unfold arch_optimized spec; simp [Signal.mux, ...]
```

This provides machine-checked guarantees that architectural optimizations preserve functional correctness—a property that simulation alone cannot ensure for all possible inputs.

4 Experiments

5 Conclusion

References

- Tutu Ajayi, David Blaauw, Timothy Chan, Chien-Yi Cheng, Vidya A Chhabria, Daniel K Choo, Matteo Coltella, Stefan Dobre, Ronald Dreslinski, Mateus Fogaça, et al. OpenROAD: Toward a self-driving, open-source digital layout implementation tool chain. *GOMAC*, 2019.
- Christiaan Baaij, Matthijs Kooijman, Jan Kuper, Arjan Boeijink, and Marco Gerards. Clash: Structural descriptions of synchronous hardware using Haskell. *EUROMICRO DSD*, pages 714–721, 2010.
- Jonathan Bachrach, Huy Vo, Brian Richards, Yunsup Lee, Andrew Waterman, Rimas Avižienis, John Wawrzynek, and Krste Asanović. Chisel: Constructing hardware in a Scala embedded language. In *DAC*, pages 1212–1221, 2012.
- Armin Biere. Bounded model checking. In *Handbook of Satisfiability*, pages 457–481. IOS Press, 2009.
- Jason Blocklove, Siddharth Garg, Ramesh Karri, and Hammond Pearce. Chip-chat: Challenges and opportunities in conversational hardware design. *MLCAD*, pages 1–6, 2023.
- Robert Brayton and Alan Mishchenko. ABC: An academic industrial-strength verification tool. In *CAV*, pages 24–40, 2010.
- Andrew Canis, Jongsok Choi, Mark Aldham, Victor Zhang, Ahmed Kammoona, Jason H Anderson, Stephen Brown, and Tomasz Czajkowski. LegUp: High-level synthesis for FPGA-based processor/accelerator systems. In *FPGA*, pages 33–36, 2011.
- Kaiyan Chang, Ying Wang, Haimeng Ren, Mengdi Wang, Shengwen Liang, Yinhe Li, Huawei Song, and Weikang Li. ChipGPT: How far are we from natural language hardware design. *arXiv preprint arXiv:2305.14019*, 2024.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Joonwon Choi, Muralidaran Vijayaraghavan, Benjamin Sherman, Adam Chlipala, and Arvind. Kami: A platform for high-level parametric hardware specification and its modular verification. In *ICFP*, pages 1–30, 2017.
- Joonwon Choi, Adam Chlipala, and Arvind Ramanathan. Silver Oak: Hardware design and verification in Coq. *arXiv preprint arXiv:2104.00038*, 2021.
- Edmund M Clarke, William Klieber, Miloš Nováček, and Paolo Zuliani. Model checking and the state explosion problem. In *TACAS*, pages 1–30, 2018.

374 Jason Cong, Bin Liu, Stephen Neuendorffer, Juanjo Noguera, Kees Vissers, and Zhiru Zhang. High-
375 level synthesis for FPGAs: From prototyping to deployment. *IEEE TCAD*, 30(4):473–491, 2011.

376 Jan Decaluwe. MyHDL: A python-based hardware description language. *Linux Journal*, 2004(127):
377 5, 2004.

378 Google and SkyWater Technology. The SkyWater open source PDK. *GitHub repository*, 2020.
379 <https://github.com/google/skywater-pdk>.

380 Warren A Hunt, Matt Kaufmann, J Strother Moore, and Anna Slobodova. Industrial hardware and
381 software verification with ACL2. In *Philosophical Transactions of the Royal Society A*, volume
382 375, page 20150399, 2017.

383 Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik
384 Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *ICLR*, 2024.

385 Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou,
386 Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. StarCoder 2 and the Stack v2: The
387 next generation. *arXiv preprint arXiv:2402.19173*, 2024.

388 Mingjie Liu, Nathaniel Pinckney, Bruce Khailany, and Haoxing Ren. VerilogEval: Evaluating
389 large language models for Verilog code generation. In *ICCAD*, pages 1–8, 2023.

390 Derek Lockhart, Gary Zibrat, and Christopher Batten. PyMTL: A unified framework for vertically
391 integrated computer architecture research. In *MICRO*, pages 280–292, 2014.

392 Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. RTLLM: An open-source benchmark for design
393 RTL generation with large language model. *ASP-DAC*, pages 722–727, 2024.

394 Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language.
395 *CADE*, pages 625–635, 2021.

396 Rishiyur S Nikhil. Bluespec SystemVerilog: Efficient, correct RTL from high level specifications.
397 In *MEMOCODE*, pages 69–70. IEEE, 2004.

398 Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri, and Siddharth Garg. DAVE:
399 Deriving automatically Verilog from English. *MLCAD*, pages 27–32, 2020.

400 Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving.
401 *arXiv preprint arXiv:2009.03393*, 2020.

402 Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi
403 Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. Code Llama: Open foundation models for
404 code. *arXiv preprint arXiv:2308.12950*, 2023.

405 Shailja Thakur, Baleegh Ahmad, Zhenxing Fan, Hammond Pearce, Benjamin Tan, Ramesh Karri,
406 Brendan Dolan-Gavitt, and Siddharth Garg. Benchmarking large language models for automated
407 Verilog RTL code generation. In *DATE*, pages 1–6, 2023.

408 Shailja Thakur, Baleegh Ahmad, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh
409 Karri, and Siddharth Garg. VeriGen: A large language model for Verilog code generation. *ACM*
410 *TODAES*, 29(3):1–31, 2024.

411 John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Liber, Shunyu Narasimhan, and Karthik
412 Narasimhan. SWE-agent: Agent-computer interfaces enable automated software engineering.
413 *NeurIPS*, 2024a.

414 Kaiyu Yang, Aidan M Swope, Alex Gu, Rahul Chalapathy, Peiyang Song, Shixing Yu, Maruan
415 Al-Shedivat, Jia Lei, Pengcheng Xia, Graham Neubig, et al. LeanDojo: Theorem proving with
416 retrieval-augmented language models. *NeurIPS*, 2024b.

417 Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao.
418 ReAct: Synergizing reasoning and acting in language models. *ICLR*, 2023.